Métodos Numéricos y Optimización
Lecture 23: Momentum

Martín Palazzo

Universidad de San Andrés
Ingeniería en Inteligencia Artificial

palazzom@udesa.edu.ar

June 19, 2024

Overview

1. Gradiente descendiente en funciones cuadráticas convexas
2. Gradiente descendiente con Momentum
3. Momentum: Análisis de aceleración sobre una función cuadrática
4. Gradiente descendiente estocástico

Gradiente descendiente en funciones cuadráticas convexas

Gradiente descendiente en función cuadrática

Analicemos la Convergencia de

$$f(x) = x_1^2 + bx_2^2$$

para distintos valores de b y para un *step-size* constante $s_k = 0.1$ para todos los casos

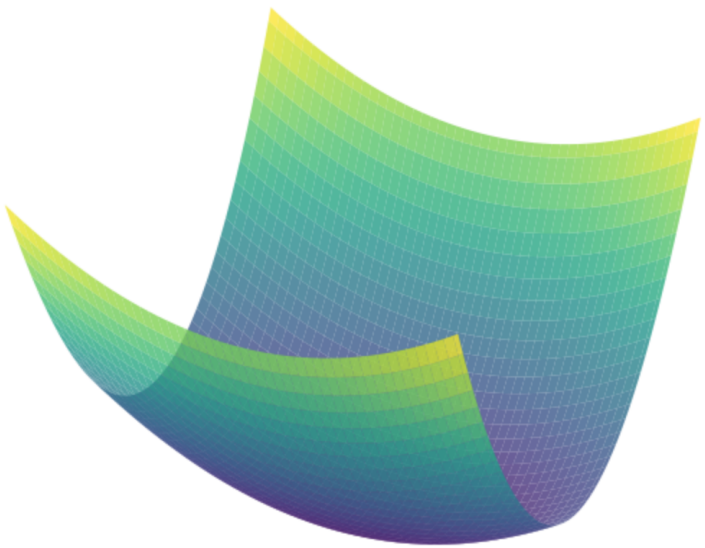


Figure: $f(x) = x_1^2 + bx_2^2$

Gradiente descendiente en función cuadrática

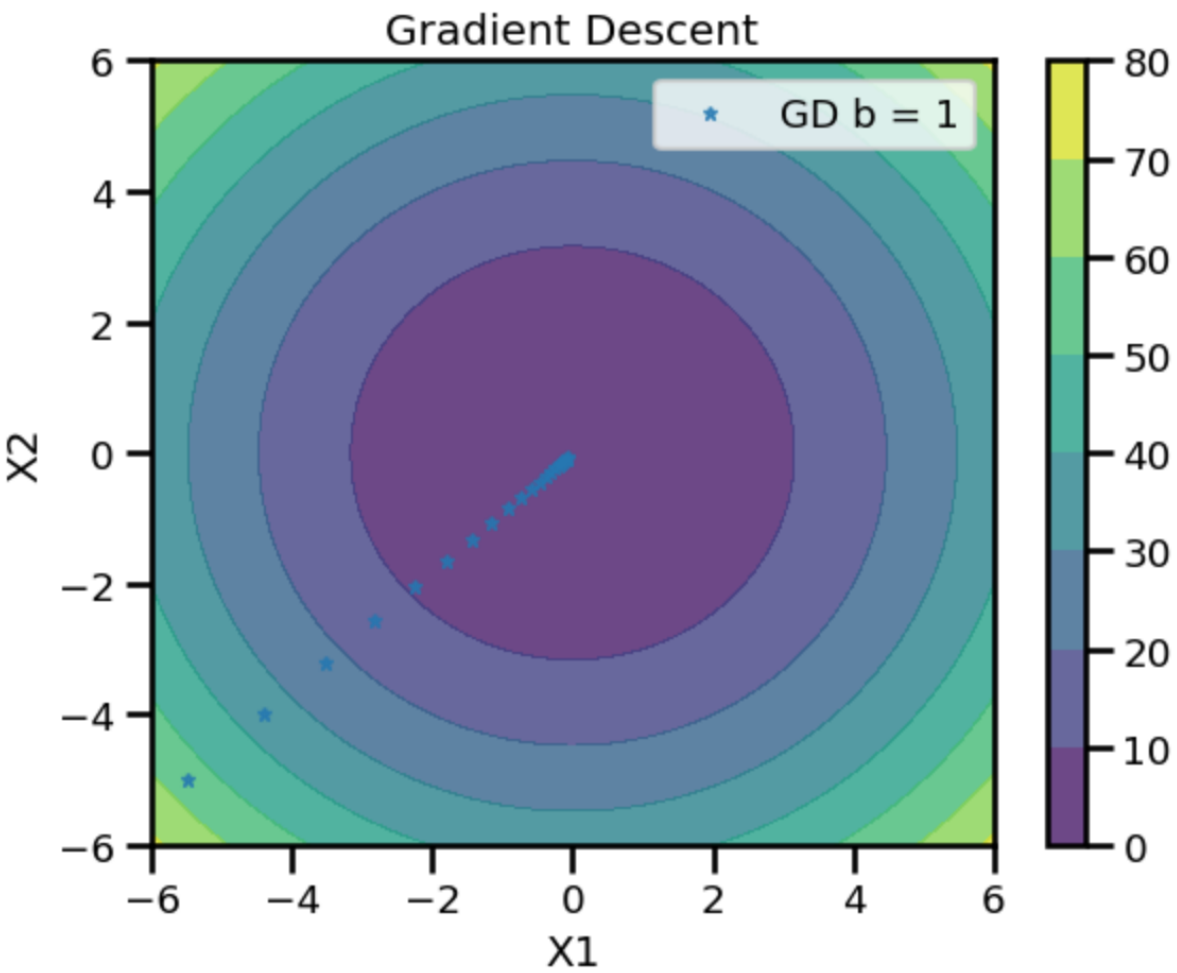


Figure: Convergencia de $f(x) = x_1^2 + bx_2^2$ con $b = 1$ y $s_k = 0.1$ constante

Gradiente descendiente en función cuadrática

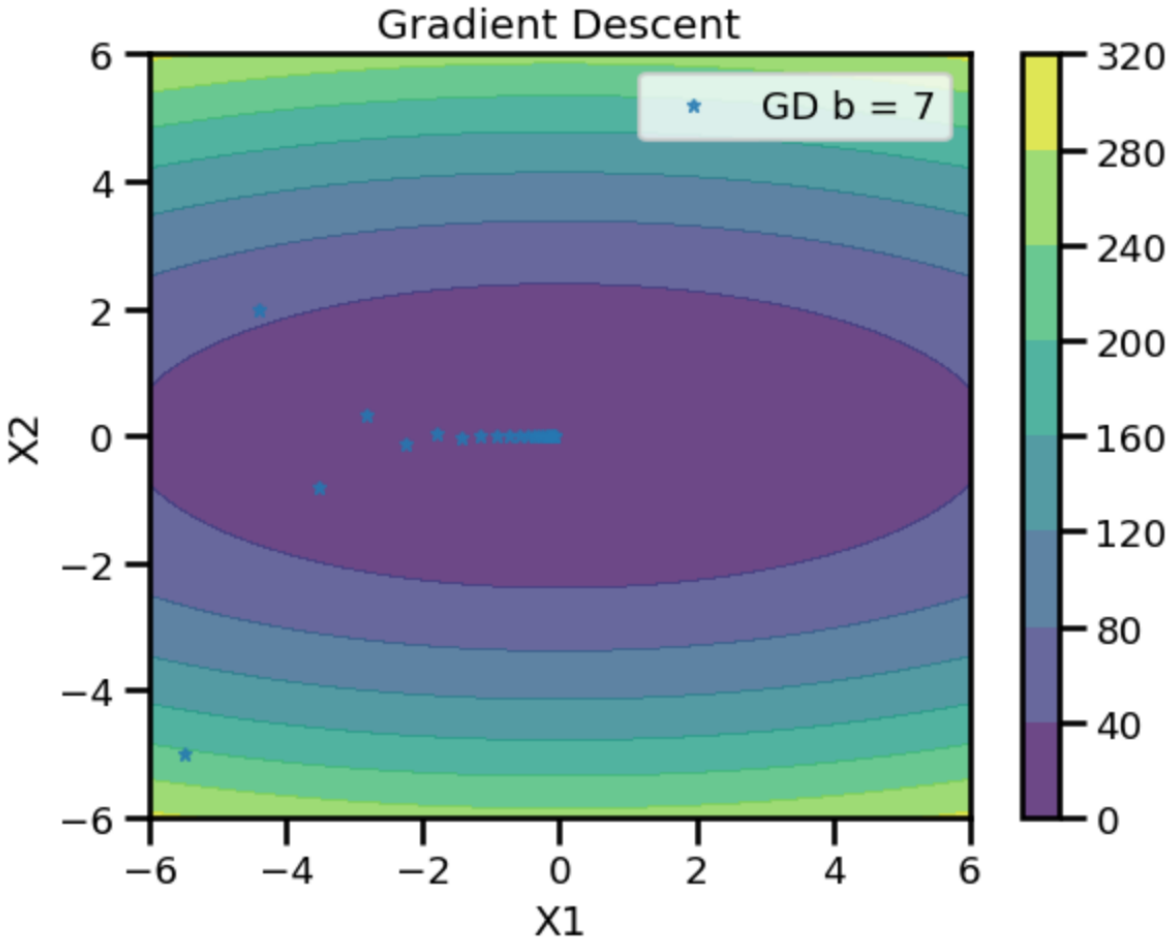


Figure: Convergencia de $f(x) = x_1^2 + bx_2^2$ con $b = 7$ y $s_k = 0.1$ constante

Gradiente descendiente en función cuadrática

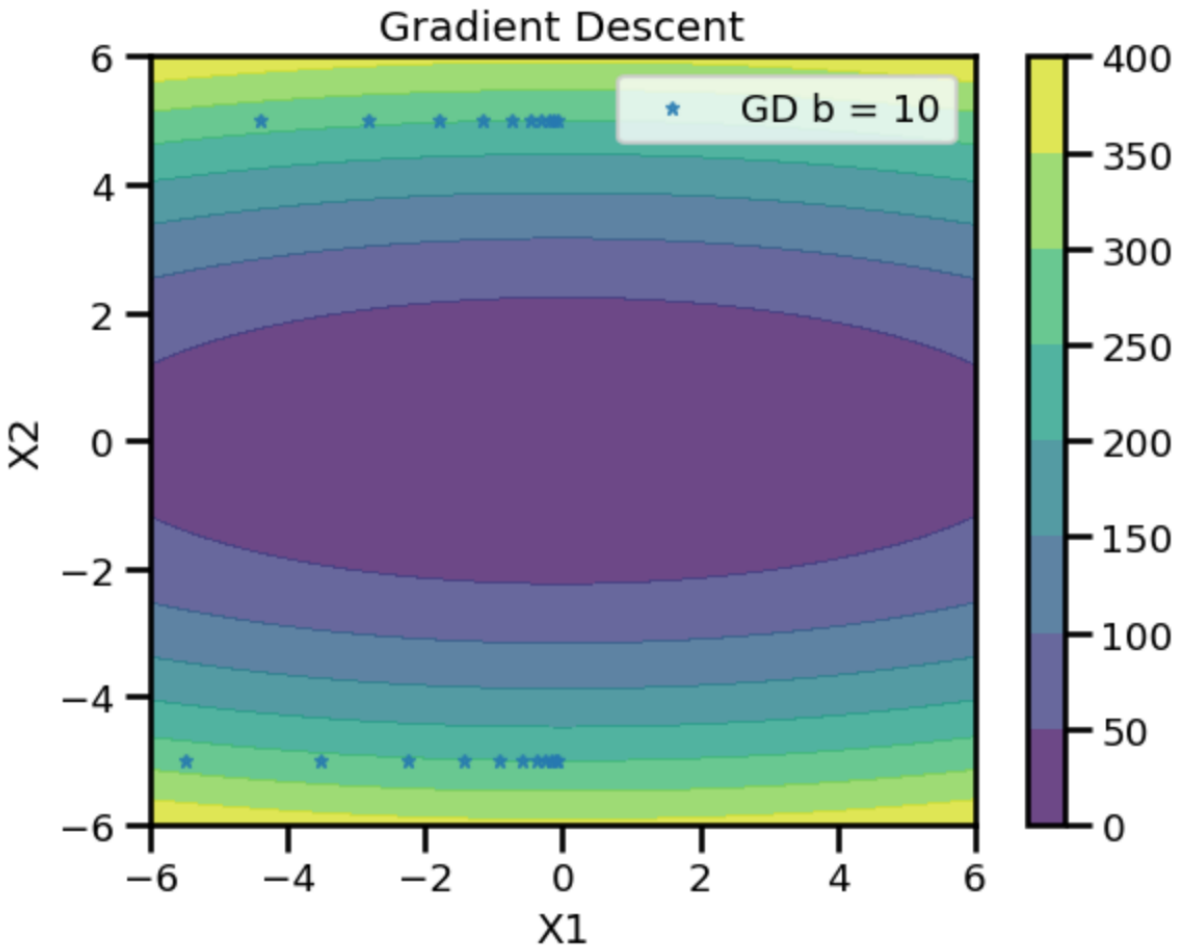


Figure: Convergencia de $f(x) = x_1^2 + bx_2^2$ con $b = 10$ y $s_k = 0.1$ constante

Gradiente descendiente en función cuadrática

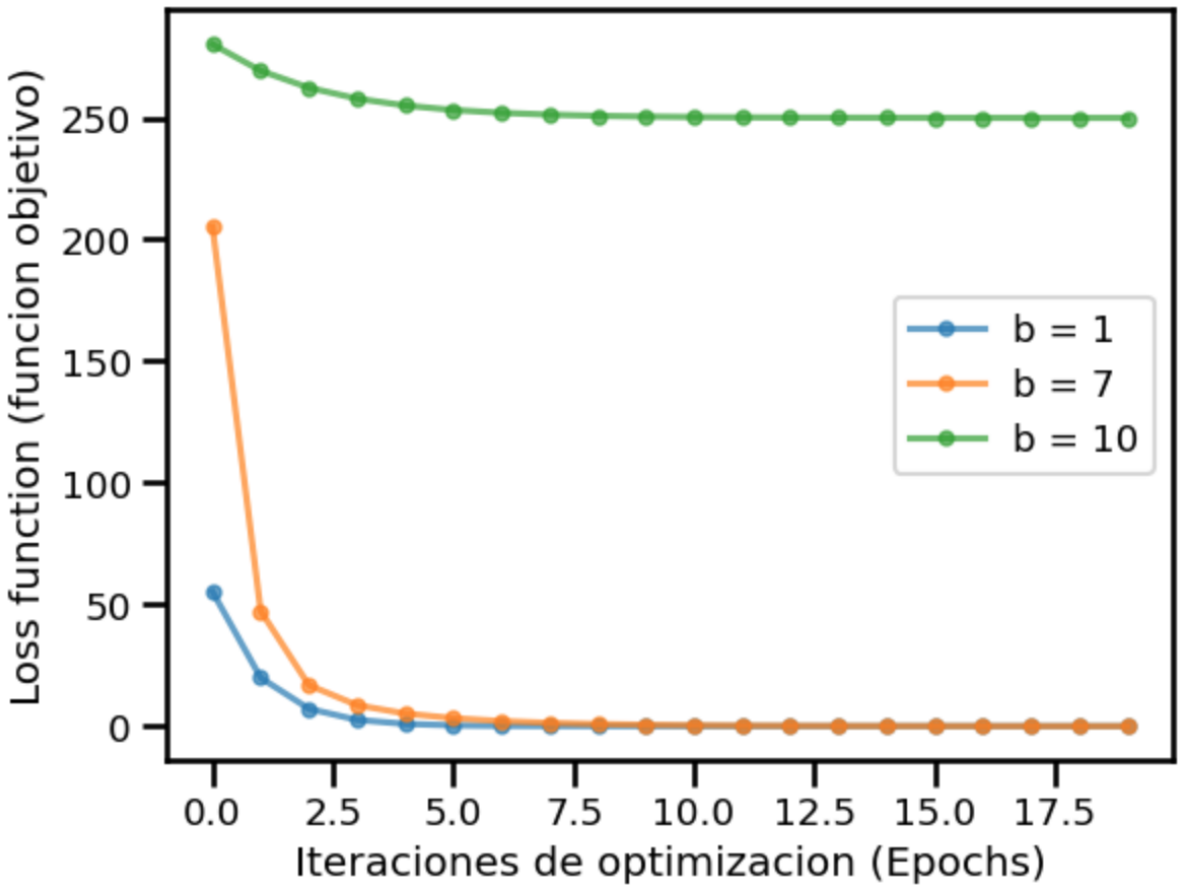


Figure: Convergencia de $f(x) = x_1^2 + bx_2^2$ para distintos valores de b y $s_k = 0.1$ constante

Gradiente descendiente con Momentum

Gradiente descendiente con Momentum

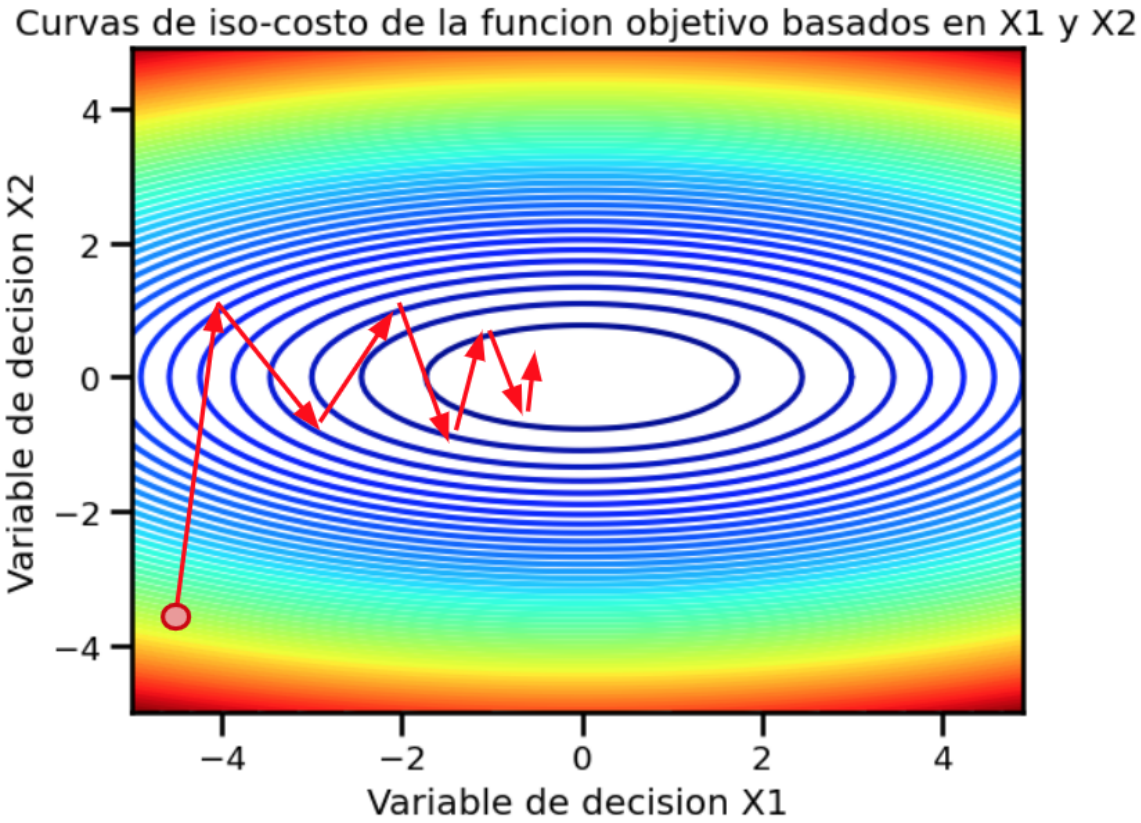


Figure: Convergencia al óptimo con zig-zag

La clase pasada habíamos concluido que el gradiente descendiente aproxima al óptimo en una función cuadrática convexa con un *zig-zag*.

Gradiente descendiente con Momentum

Limitaciones del Gradiente descendiente

El *zig-zag* (es decir, siempre avanzando perpendicularmente a las lineas de nivel de la funcion objetivo) genera una convergencia lineal que puede ser mejorada.

Pathological curvature

En ciertas regiones del dominio la función objetivo puede presentar *Pathological Curvatures*. Es decir una región que la curvatura no propicia un avance importante en la minimización de la función objetivo.

Gradiente descendiente con Momentum

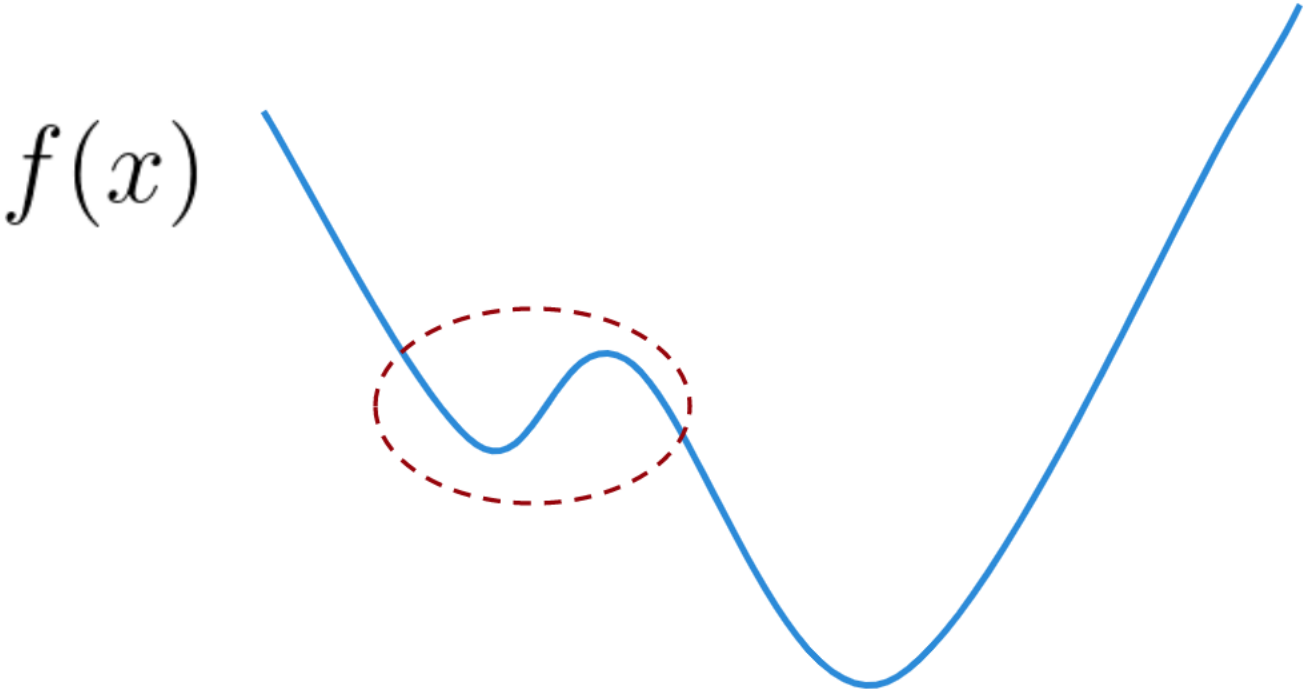


Figure: Un caso de alta curvatura puede resultar con GD en un mínimo local [4].

Gradiente descendiente con Momentum

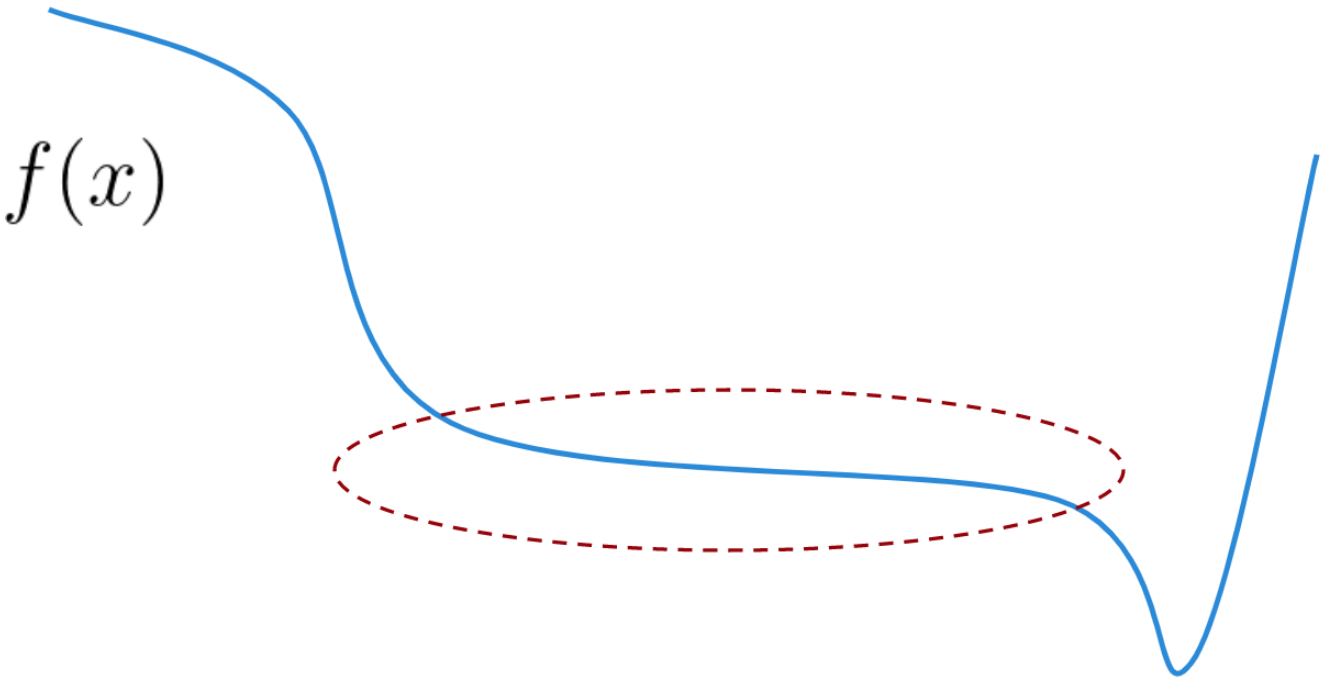


Figure: Un caso de bajo gradiente puede demorar la convergencia al mínimo global aun en funciones convexas! [4]

Gradiente descendiente con Momentum

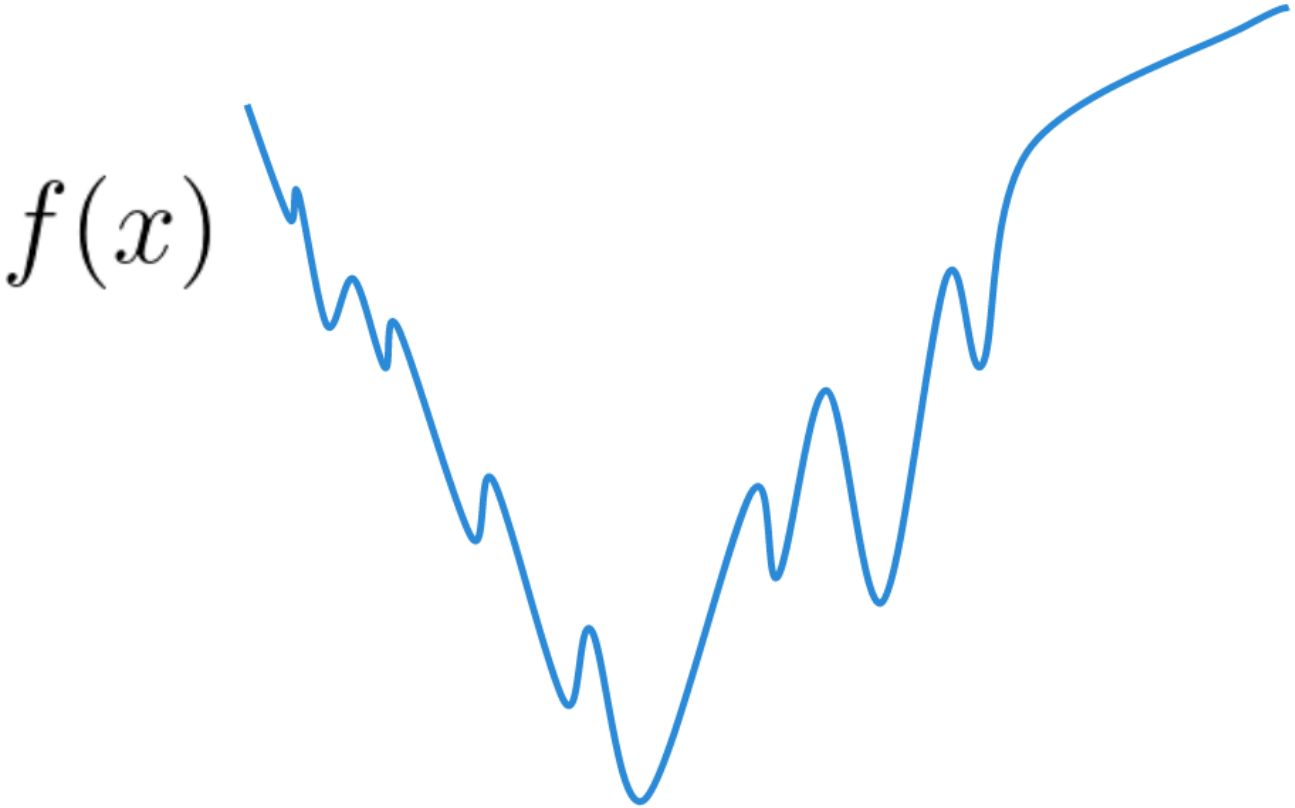


Figure: El gradiente afectado por ruido también dificulta la convergencia del método de optimización al mínimo global [4]

Gradiente descendiente con Momentum

Momentum

De la clase pasada la función que estudiamos

$$f = \frac{1}{2}(x_1^2 + bx_2^2)$$

con Hessiano H donde si

$$b = \lambda_{min}/\lambda_{max} = m/M$$

es chico entonces el *zig-zag* es mas intenso. λ son los autovalores de H.

Gradiente descendiente con Momentum

Momentum

Para evitar el *zig-zag* se busca agregarle *inercia* al gradiente descendiente (conocido como *Heavy Ball*) agregando el parámetro β de Momentum al gradiente

$$x_{k+1} = x_k - s z_k$$

donde la dirección

$$z_k = \nabla f(x_k) + \beta z_{k-1}$$

del próximo paso tienen memoria de la dirección del paso anterior z_{k-1}

Gradiente descendiente con Momentum

Gradiente descendiente con momentum

Hay dos coeficientes a determinar

- s el *stepsize*
- β el momentum

Para calcular la solución en el próximo paso x_{k+1} hay que calcular z_{k-1}, es decir que Momentum transforma al Gradiente Descendiente en método de 1 paso a uno de 2 pasos en cada iteración x_k. Para el estado $k+1$

$$\begin{aligned} x_{k+1} &= x_k - s z_k \\ z_{k+1} &= \nabla f(x_{k+1}) + \beta z_k \end{aligned} \tag{1}$$

Momentum: Análisis de aceleración sobre una función cuadrática

Gradiente descendiente con Momentum

Sigamos analizando la función cuadrática (convexa)

$$f = \frac{1}{2}(x_1^2 + bx_2^2)$$

que si generalizamos la expresión de la cuadrática como

$$f(x) = \frac{1}{2}x^T Sx$$

su gradiente lineal

$$\nabla f(x) = Sx$$

donde S es una matriz simetrica positiva definida y

$$\nabla f(x_{k+1}) = Sx_{k+1}$$

donde aplica a la ecuacion (1) slide anterior

Gradiente descendiente con Momentum

suponiendo solo 2 dimensiones podemos estudiar el modelo que generaliza a mas dimensiones. Con

$$f = \frac{1}{2}(x_1^2 + bx_2^2)$$

su gradiente S es una matriz diagonal

$$S = \begin{bmatrix} 1 & 0 \\ 0 & b \end{bmatrix}$$

basandonos en la clase anterior, los mejores valores de s y β quedaran definidos por el autovalor mas grande λ_{max} y mas chico λ_{min} de S

Gradiente descendiente con Momentum

Sabiendo que la matriz S la podemos factorizar $S\mathbf{q} = \lambda\mathbf{q}$ vamos a suponer que

$$x_k = c_k\mathbf{q} \quad , \quad z_k = d_k\mathbf{q} \quad , \quad \nabla f_k = Sx_k = \lambda c_k\mathbf{q}$$

además basándonos en las ecuaciones de Momentum vistas anteriormente

$$\begin{aligned} x_{k+1} &= x_k - sz_k \\ z_{k+1} - \nabla f(x_{k+1}) &= \beta z_k \end{aligned}$$

podemos relacionar los coeficientes c_k y d_k en un paso k con c_{k+1} y d_{k+1} en el paso $k+1$ tal que si seguimos la dirección del autovector $\mathbf{q}$

$$\begin{aligned} c_{k+1} &= c_k - sd_k \\ -\lambda c_{k+1} + d_{k+1} &= \beta d_k \end{aligned}$$

Gradiente descendiente con Momentum

de la slide anterior

$$\begin{aligned} c_{k+1} &= c_k - s d_k \\ -\lambda c_{k+1} + d_{k+1} &= \beta d_k \end{aligned}$$

matricialmente

$$\begin{bmatrix} 1 & 0 \\ -\lambda & 1 \end{bmatrix} \begin{bmatrix} c_{k+1} \\ d_{k+1} \end{bmatrix} = \begin{bmatrix} 1 & -s \\ 0 & \beta \end{bmatrix} \begin{bmatrix} c_k \\ d_k \end{bmatrix}$$

Gradiente descendiente con Momentum

de la slide anterior

$$\begin{bmatrix} 1 & 0 \\ -\lambda & 1 \end{bmatrix} \begin{bmatrix} c_{k+1} \\ d_{k+1} \end{bmatrix} = \begin{bmatrix} 1 & -s \\ 0 & \beta \end{bmatrix} \begin{bmatrix} c_k \\ d_k \end{bmatrix}$$

si invertimos la primer matriz ($-\lambda$ pasa a ser $+\lambda$)

$$\begin{bmatrix} c_{k+1} \\ d_{k+1} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ \lambda & 1 \end{bmatrix} \begin{bmatrix} 1 & -s \\ 0 & \beta \end{bmatrix} \begin{bmatrix} c_k \\ d_k \end{bmatrix} = \begin{bmatrix} 1 & -s \\ \lambda & \beta - \lambda s \end{bmatrix} \begin{bmatrix} c_k \\ d_k \end{bmatrix}$$

y reescribimos $\begin{bmatrix} 1 & -s \\ \lambda & \beta - \lambda s \end{bmatrix} = \mathbf{R}$ entonces

$$\begin{bmatrix} c_{k+1} \\ d_{k+1} \end{bmatrix} = \mathbf{R} \begin{bmatrix} c_k \\ d_k \end{bmatrix}$$

Gradiente descendiente con Momentum

de la slide anterior

$$\begin{bmatrix} c_{k+1} \\ d_{k+1} \end{bmatrix} = \mathbf{R} \begin{bmatrix} c_k \\ d_k \end{bmatrix}$$

observamos que luego de k pasos el vector inicial es multiplicado por R^k. Para mayor convergencia al cero (mínimo de la cuadrática): si minimizamos el mayor de los autovalores e_1, e_2 de $\mathbf{R}$ se acelerará la convergencia (que en $f(x)$ cuadrática convexa tiene solución $x_1, x_2 = 0, 0$).

$$\begin{bmatrix} 1 & -s \\ \lambda & \beta - \lambda s \end{bmatrix} = \mathbf{R}$$

Los autovalores e_1, e_2 de R dependen de los autovalores de S y $\lambda_{min}(S) < \lambda < \lambda_{max}(S)$.

Gradiente descendiente con Momentum

Esto se resume en encontrar los mejores s_k (step) y β que minimizan el mayor autovalor

$$\underset{s,\beta}{\textbf{minmax}}\,[e_1(\lambda), e_2(\lambda)]$$

$$\text{sujeto a}$$

$$\lambda_{min}(S) \leq \lambda \leq \lambda_{max}(S)$$

y como la matriz **R** se escribe

$$\begin{bmatrix} 1 & -s \\ \lambda & \beta - \lambda s \end{bmatrix} = \mathbf{R}$$

entonces se puede llegar a la expresión de los mejores parámetros que resultan en la mejor convergencia

$$s_k = \left(\frac{2}{\sqrt{\lambda_{\max}} + \sqrt{\lambda_{\min}}}\right)^2 \beta = \left(\frac{\sqrt{\lambda_{\max}} - \sqrt{\lambda_{\min}}}{\sqrt{\lambda_{\max}} + \sqrt{\lambda_{\max}}}\right)^2$$

Gradiente descendiente con Momentum

si usamos el caso de la clase pasada donde $\lambda_{min} = b$ y $\lambda_{max} = 1$ para $f = \frac{1}{2}(x_1^2 + bx_2^2)$ entonces los parametros de GD con momentum quedan

$$s = \left(\frac{2}{1+\sqrt{b}}\right)^2 \quad \beta = \left(\frac{1-\sqrt{b}}{1+\sqrt{b}}\right)^2$$

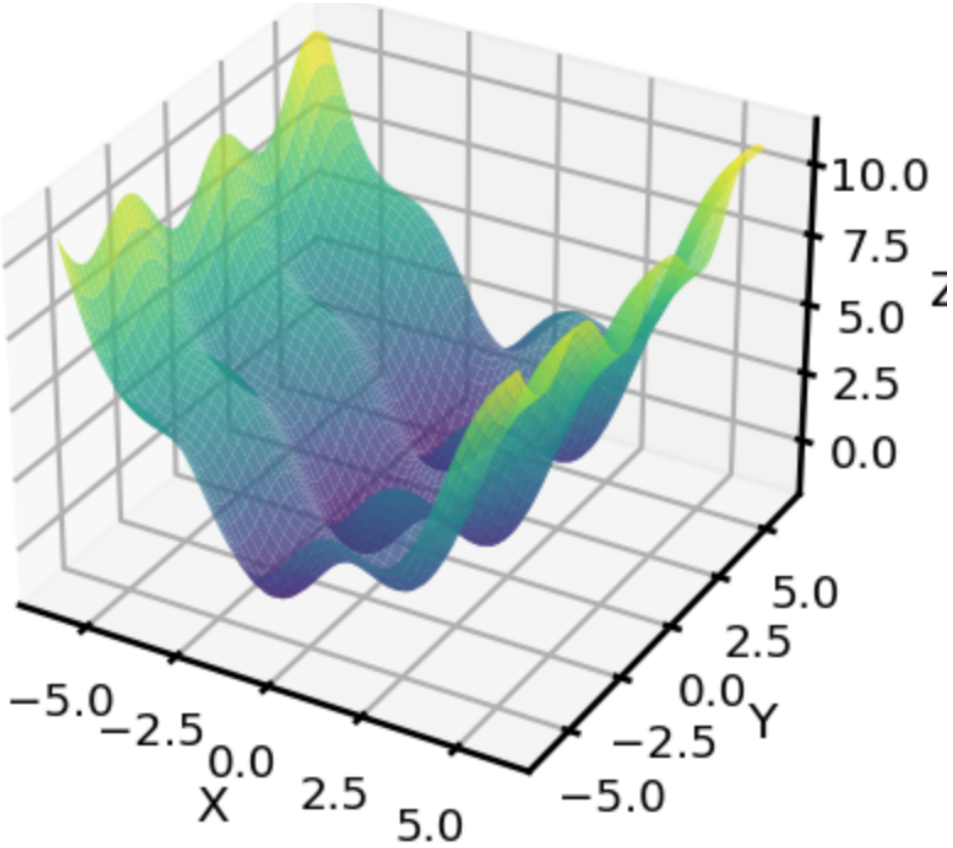
que es muy parecido a la convergencia estudiada en la sección de interpretación geométrica de la clase pasada

$$f\left(x_{1(k)}, x_{2(k)}\right) = \left(\frac{1-b}{1+b}\right)^{2k} f\left(x_{1(k=0)}, x_{2(k=0)}\right)$$

con la diferencia de que ahora en vez de tener b, con momentum tenemos $\sqrt{b}$!! es decir que si b muy chico ej. $b = 1/100$ entonces $\sqrt{b} = 1/10$ quedando la convergencia

$$\text{GD} \rightarrow \left(\frac{0.99}{1.01}\right)^2 = 0.96 \quad \text{Momentum} \rightarrow \left(\frac{0.9}{1.1}\right)^2 = 0.67$$

Gradiente descendiente con Momentum



supongamos la siguiente función cuyo gradiente podemos calcular

$$f(x_1, x_2) = \sin(0.5\pi x_1) + 0.25x_1^2 + \sin(0.5\pi x_2) + 0.05x_2^2$$

Gradiente descendiente con Momentum

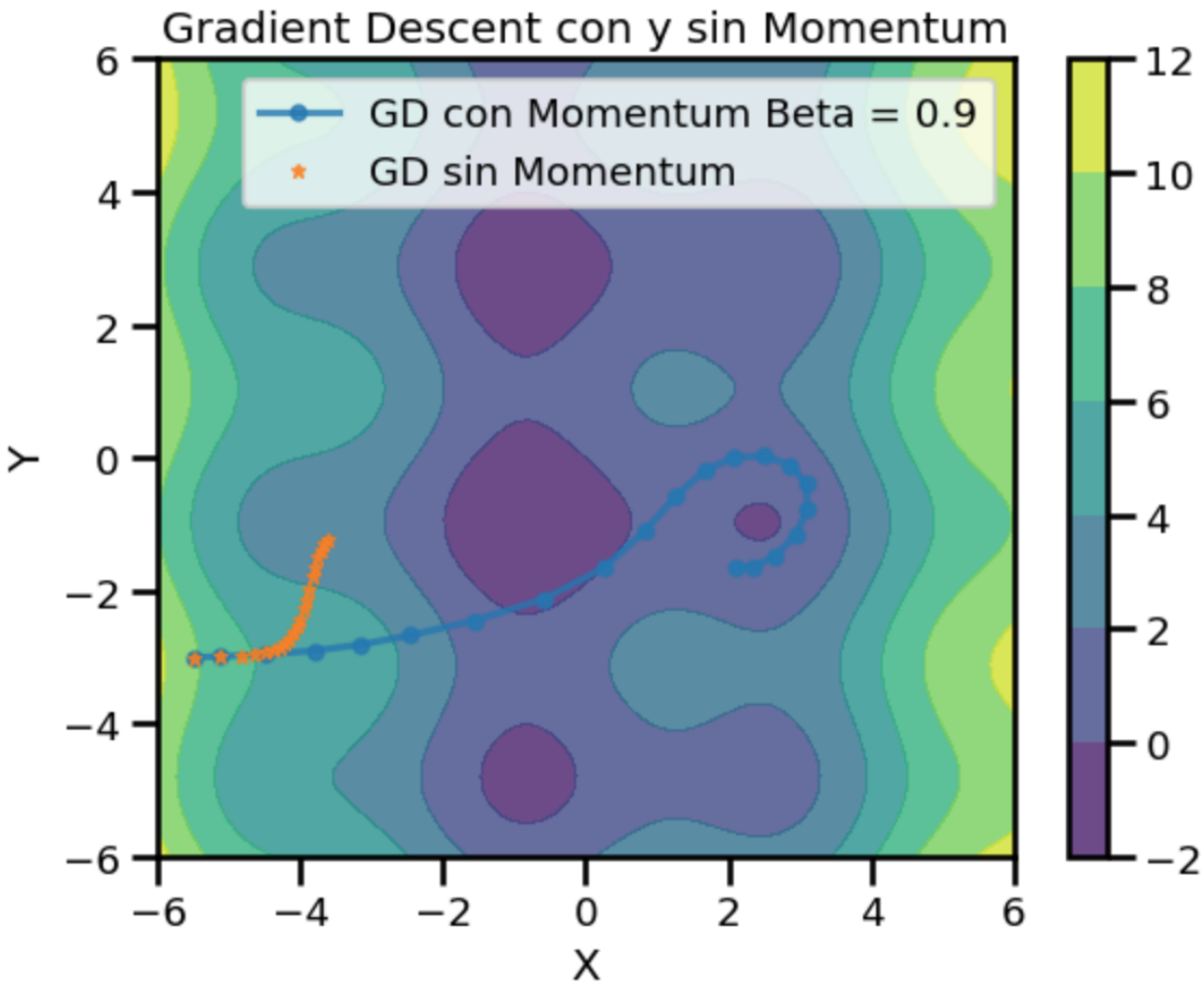


Figure: Momentum $\beta = 0.9$, $k = 20$

Gradiente descendiente con Momentum

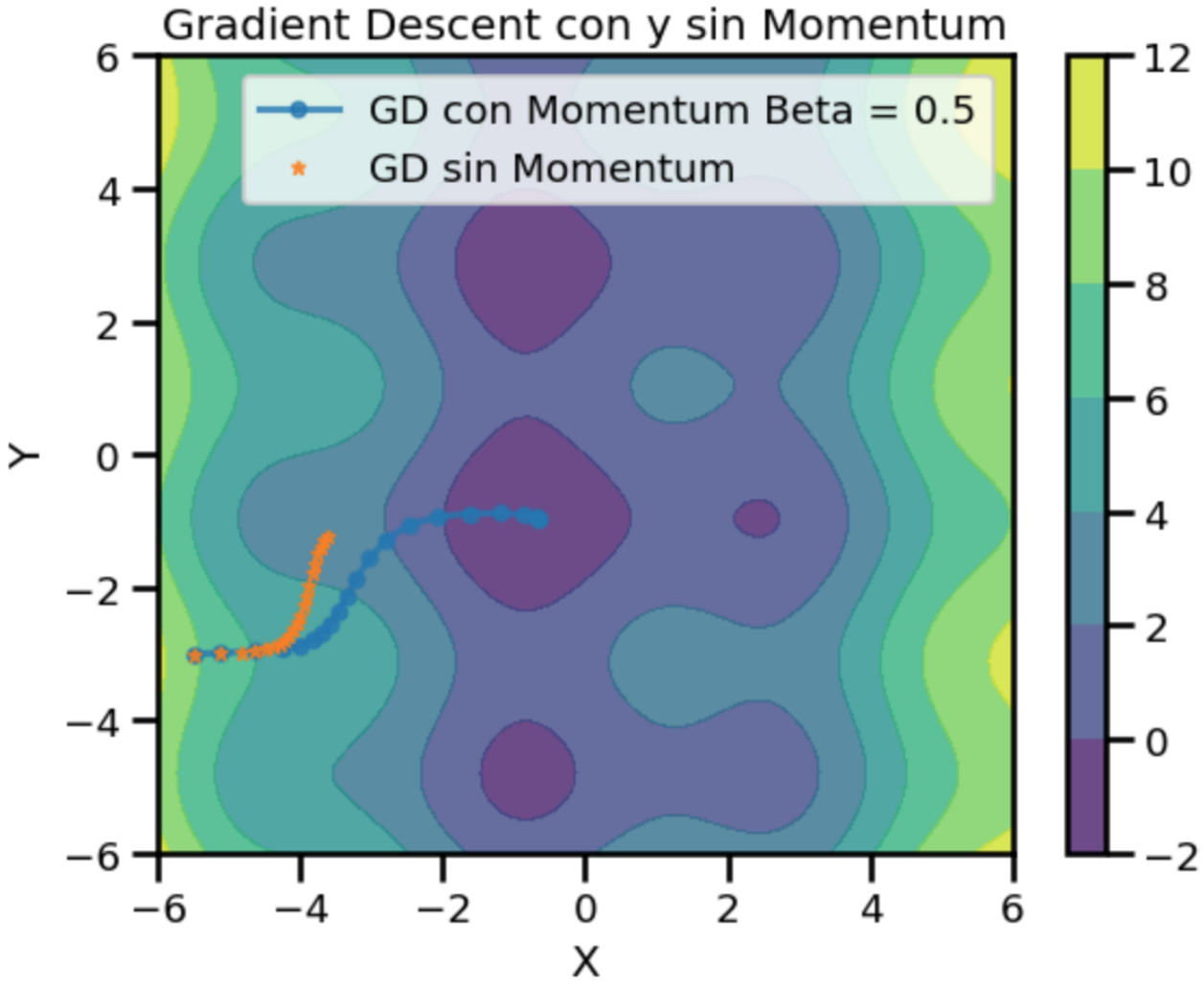


Figure: Momentum $\beta = 0.5,\ k = 20$

Gradiente descendiente con Momentum

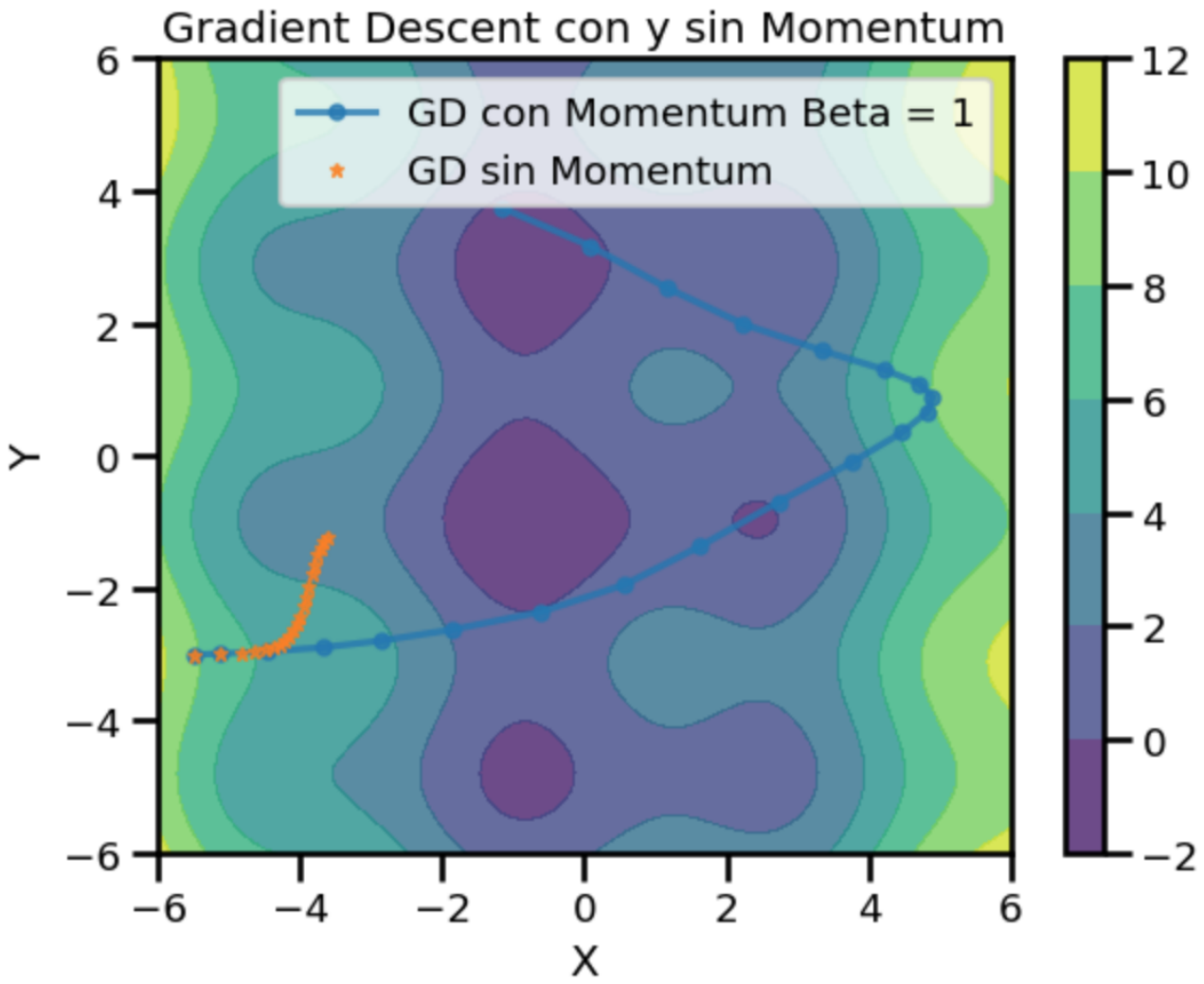


Figure: Momentum $\beta = 1,\ k = 20$

Gradiente descendiente con Momentum

Slide dedicada a recordarles que lean las siguientes referencias

- *Why Momentum works* [3]
- Momentum: *Linear algebra and learning from data* capitulo VI.4 [2]
- Momentum: *Deep Learning book* capítulo 8 [4]

Gradiente descendiente estocástico

Gradiente descendiente estocástico

En optimización algunas funciones de costo a minimizar con x como variable de decisión pueden expresarse como

$$F(x) = \frac{1}{N}\sum_{i=1}^{N} F_i(x)$$

en el caso de una función cuadrática con $x \in \mathbb{R}^n$ como variable de decisión

$$F(x) = \frac{1}{2}x^\top Sx = \sum_{i=1}^{N}\left(\frac{1}{2}\sum_{j=1}^{N} S_{ij}x_i x_j\right)$$

o en cuadrados mínimos

$$F(x) = \frac{1}{N}\sum_{i=1}^{N}\left(y_i - \hat{f}\left(w_i, x\right)\right)^2$$

donde ahora w es el vector incógnita a averiguar

Gradiente descendiente estocástico

Motivación de SGD: Stochastic Gradient Descent

Cuando n es muy grande (muchas muestras) o la dimensionalidad del vector de las variables de decisión $x \in \mathbb{R}^d$ es muy grande calcular el gradiente $\nabla f(x)$ puede llegar a ser muy caro.

Gradiente descendiente estocástico

$$S : \{x_1, x_2, x_3, ..., x_n\}$$

$$\mathbf{X}_{(n,d)} = \begin{bmatrix} x_{00} & x_{01} & ... & x_{0d} \\ x_{10} & x_{11} & ... & x_{1d} \\ ... & ... & ... & ... \\ x_{n0} & x_{n1} & ... & x_{nd} \end{bmatrix}$$

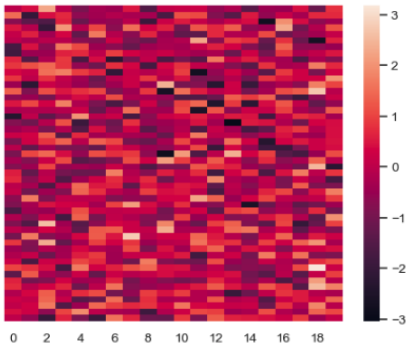


Figure: Los datos suelen presentarse como matrices. Podemos usar la matriz completa para computar los gradientes aunque esto puede ser computacionalmente caro dada la alta dimensión de los problemas contemporaneos.

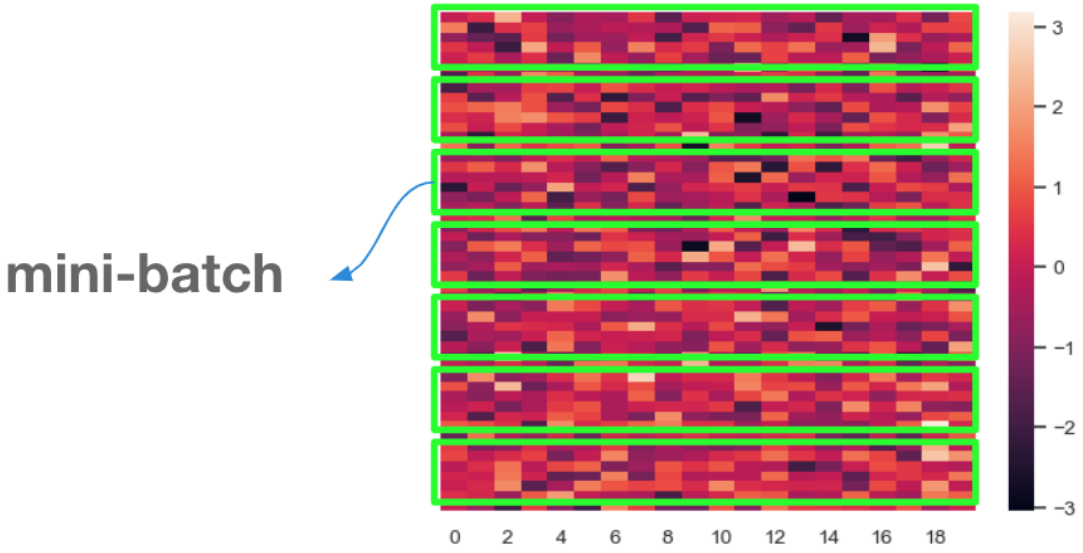
Gradiente descendiente estocástico

SGD: minibatch

Lo que podemos hacer entonces es construir *mini-batchs* de $B = \mathbf{v}$ muestras

$$f(x) = \frac{1}{n} \sum_{i}^{n} f_i(x)$$

un caso comun es mini-batch de $B = 1$



Gradiente descendiente estocástico

Stochastic Gradient Descent

computamos el gradiente usando un mini-batch obtenido de manera aleatoria y actualizamos la solución

$$x_{k+1} = x_k - s_k \nabla_x f_i(x_k)$$

es decir que si tenemos 100 muestras y el mini-batch es $B = 1$ ordenaremos aleatoriamente las muestras y por cada una calcularemos el gradiente y actualizar la solución para minimizar la función. Una vez que pasemos por todas las muestras realizaremos un **Epoch**. Podemos pasar varias epochs las muestras para ir refinando la solución, siempre con un orden aleatorio distinto.

Gradiente descendiente estocástico

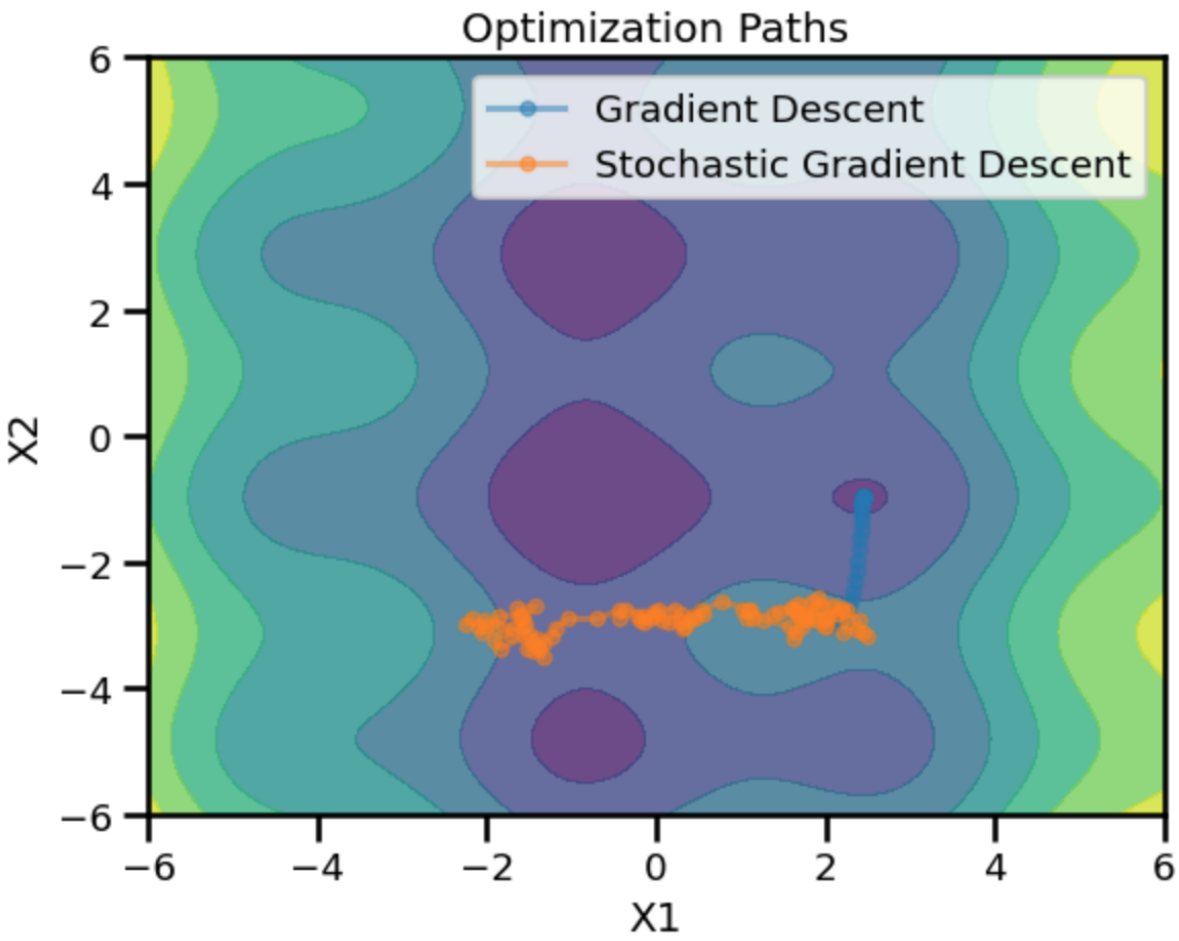


Figure: El hecho de usar un subset de muestras para computar el gradiente hace que no sea tan directo en llegar al óptimo, de todos modos en alta dimensión es una propuesta escalable.

Gradiente descendiente estocástico

en el caso de cuadrados minimos

$$F(x) = \|Ax - b\|^2 = \sum_{i=1}^{N} (a_{ij}x_j - b_i)^2$$

en vez de resolver todo junto, podríamos tomar una fila de la matriz A (dato) e ir resolviendo una ecuación a la vez y en cada iteración ir actualizando la solución x (incógnita).

Gradiente descendiente estocástico

Por que lo usamos?

En Deep Learning (machine learning con redes neuronales) vamos a querer resolver un problema de optimización que minimice la función objetivo de *Empirical Risk Minimization*

$$\min_w L(y, \hat{y}) = \min_w L(y, f_w(x))$$

si el dataset X es de alta dimension y con muchas muestras va a costar computar el gradiente. Además no vamos a querer *sobre-ajustar* a ese dataset ya que queremos generalizar a muestras no vistas. Obviamente con Stochastic Gradient Descent la optimización será ruidosa y no tan limpia como la del GD standard.

ADAM: adaptative moment estimation

ADAM optimizer

Combina la idea de momentum de usar gradientes de pasos anteriores y SGD

$$x_{k+1} = x_k - s_k D_k$$

donde el *stepsize* y la solución en el paso k

$$\begin{aligned} D_k =& \delta D_{k-1} + (1-\delta)\nabla f_{i(k)}(x_k) \\ S_k^2 =& \beta S_{k-1}^2 + (1-\beta)\|\nabla f_{i(k)}(x_k)\|^2 \end{aligned}$$

Nocedal, J., & Wright, S. J. (Eds.). (1999). Numerical optimization. New York, NY: Springer New York.

Strang, G. (2019). Linear algebra and learning from data (Vol. 4). Cambridge: Wellesley-Cambridge Press.

Goh, G. (2017). Why momentum really works. Distill, 2(4), e6.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT press.